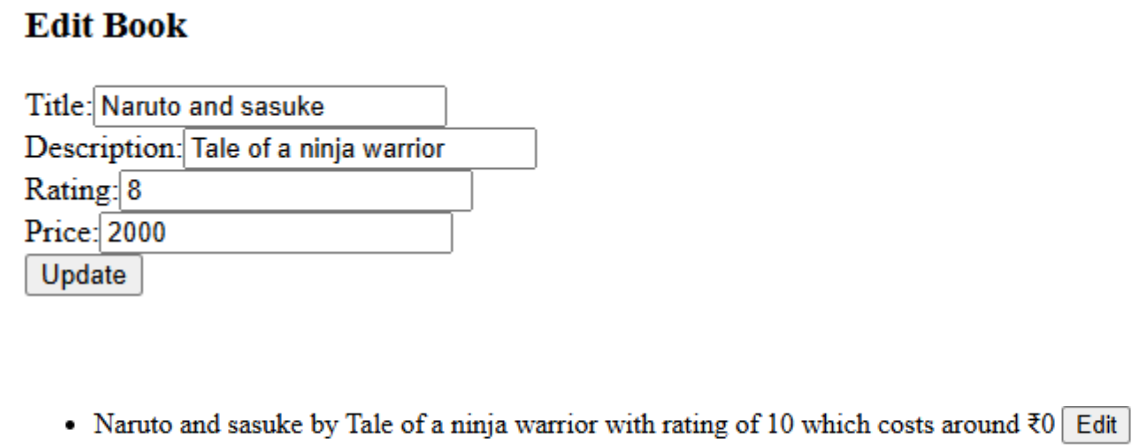
DAILY TASK 17

NAME : SANJEEV
DATE : 29-08-2025




```
<input id="nameInput" type="text" [(ngModel)]="newPlace"
placeholder="Place Name">
<input id="altInput" type="number" [(ngModel)]="altplace"
placeholder="Altitude">
<button (click)="addmore()">Add place</button>

<ul>
  <li *ngFor="let a of my_fav_places">
    {{ a.Name }} is at {{ a.altitude }} m
  </li>
</ul>

<div>
  <input type="text" [(ngModel)]="newBookTitle" placeholder="Title">
  <input type="text" [(ngModel)]="newBookDescription"
placeholder="Description">
  <input type="number" [(ngModel)]="newBookRating" placeholder="Rating">
  <input type="number" [(ngModel)]="newBookPrice" placeholder="Price">
  <button (click)="addBook()">Add Book</button>
</div>
```

```
<div>
  <input type="number" [(ngModel)]="deleteBookId" placeholder="Enter Book
ID to delete">
  <button (click)="deleteBook()">Delete Book</button>
</div>

<ul>
  <li *ngFor="let b of favBook">
    <!-- Display mode -->
    <span *ngIf="selectedBook?.bookId !== b.bookId">
      {{ b.title }} by {{ b.description }} with rating of {{ b.rating }}
which costs around ₹{{ b.price }}
      <button (click)="selectBook(b)">Edit</button>
    </span>

    <!-- Edit mode -->
    <span *ngIf="selectedBook?.bookId === b.bookId">
      <input [(ngModel)]="selectedBook!.title" placeholder="Title">
      <input [(ngModel)]="selectedBook!.description"
placeholder="Description">
      <input type="number" [(ngModel)]="selectedBook!.rating"
placeholder="Rating">
      <input type="number" [(ngModel)]="selectedBook!.price"
placeholder="Price">
      <button (click)="updateBook()">Update</button>
      <button (click)="selectedBook = undefined">Cancel</button>
    </span>
  </li>
</ul>




<div *ngIf="selectedBook">
  <h3>Edit Book</h3>
  <form #editForm="ngForm">
    <label>Title:</label>
```

```
    <input [(ngModel)]="selectedBook!.title" name="title"
placeholder="Title"><br>

    <label>Description:</label>
    <input [(ngModel)]="selectedBook!.description" name="description"
placeholder="Description"><br>

    <label>Rating:</label>
    <input type="number" [(ngModel)]="selectedBook!.rating" name="rating"
placeholder="Rating"><br>

    <label>Price:</label>
    <input type="number" [(ngModel)]="selectedBook!.price" name="price"
placeholder="Price"><br>

    <button type="button" (click)="updateBook()">Update</button>
  </form>
</div>
```

```
import { Component } from '@angular/core';
import { place } from 'src/models/place.interface';
import { BookService } from '../book.service';
import { Book } from 'src/models/books.interface';
import { entry } from 'src/models/entry.interface';
import { NewBook } from 'src/models/newBook.interface';
@Component({
  selector: 'app-test',
  templateUrl: './test.component.html',
  styleUrls: ['./test.component.css']
})


export class TestComponent {
  constructor(private bookService: BookService)
  {

  }
```

```
ooty:place = {Name:'Leh Ladakh', altitude: 2240}
newPlace : string = ""
altplace : number = 0
favBook: Book[] = [];
newBookTitle: string = "";
newBookDescription: string = "";
newBookRating: number = 0;
newBookPrice: number = 0;
deleteBookId: number = 0;
allBooks: Book[] = [];
selectedBook?: Book;



addBook() {
  console.log("Started");

  const newEntry: entry = {
    title: this.newBookTitle,
    description: this.newBookDescription,
    rating: this.newBookRating,
    price: this.newBookPrice
  };

  this.bookService.addbook(newEntry).subscribe((res) => {
    this.favBook = [...this.favBook, newEntry];
  }

 );
this.loadBooks();
}

selectBook(book: Book) {
  console.log("tetvdvchduahjebud")
  this.selectedBook = { ...book };
}

updateBook() {
  if (!this.selectedBook || this.selectedBook.bookId === undefined)
return;
```

```
  this.bookService.updateBook(this.selectedBook.bookId, this.selectedBook)
    .subscribe(res => {
      console.log('Book updated successfully', res);
      this.loadBooks();
      this.selectedBook = undefined;
    });
}


loadBooks() {
  this.bookService.getfavbook().subscribe((res: Book[]) => {
    this.favBook = res;
    this.allBooks = res;
  });
}




addmore()
{
  console.log("clicked")
  this.my_fav_places.push({Name:'Ooty', altitude: 1200})
  this.my_fav_places = [...this.my_fav_places,{Name : this.newPlace,
altitude : this.altplace}]

}
my_fav_places: place[]  = [
  {Name : 'Valparai', altitude: 1800},
  {Name : 'Munnar', altitude: 2000},
  {Name : 'Dhimbam', altitude: 1500}
]
login() {
    this.bookService.login();
}

loginAndLoadBooks() {
  this.bookService.login().subscribe(token => {
    console.log("Received token:", token);
```

```
    localStorage.setItem('token',token);
    this.bookService.setToken(token);
    this.loadBooks();
  });
}

deleteBook() {
  if (this.deleteBookId) {
    this.bookService.deleteBook(this.deleteBookId).subscribe({
      next: () => {
        console.log(`Book with ID ${this.deleteBookId} deleted`);
        this.loadBooks();
        this.deleteBookId = 0;
      }
    });
  }
}

loadAllBooks() {
  this.bookService.getAllNewBooks().subscribe((data: NewBook[]) => {
    // this.allBooks = data;
    console.log("Fetched nested books:", this.allBooks);
  });
}



place_fr_ser :string = ""
my_place:place[] = []
// ngOnInit()
// {
//   // this.my_place = this.bookService.getfavplaces()
//   // console.log( `init ${this.my_place}`)


// console.log("before")
// this.favBook = this.bookService.getfavbook();
//     console.log("favBook from service:", this.favBook);
// }
```

```
ngOnInit() {
//    console.log("before");

//    this.bookService.getfavbook().subscribe((res: Book[]) => {
//      this.favBook = res;
//      console.log("favBook :", this.favBook);
//    });
this.loadBooks();
this.loadAllBooks();
}


ngAfterContentInit()
{
  console.log("After content Initialized")
}
}
```

```
import { Injectable } from '@angular/core';
import {
  HttpRequest,
  HttpHandler,
  HttpEvent,
  HttpInterceptor
} from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class AuthorInterceptor implements HttpInterceptor {

  constructor() {}

  intercept(request: HttpRequest<unknown>, next: HttpHandler):
Observable<HttpEvent<unknown>> {
    const token_stored = localStorage.getItem("token")
    request = request.clone({headers: request.headers.set
('Authorization','bearer '+ token_stored)})
    console.log("added the jwt Token")
```

```
    return next.handle(request);
  }
}
```

```
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { Book } from 'src/models/books.interface';
import { entry } from 'src/models/entry.interface';
import { NewBook } from 'src/models/newBook.interface';
import { place } from 'src/models/place.interface';
@Injectable({
  providedIn: 'root'
})
export class BookService {

  private apiURL = "http://localhost:5056/Book";
  private Login = "http://localhost:5056/User";
  private token: string = "";

login(username: string = "sanjeev", password: string = "sanjeev"){
  return this.http.post(`${this.Login}/Login`, { name: username, password
}, { responseType: 'text' });
}

setToken(token: string) {
  this.token = token;
}

  constructor(private http : HttpClient) { }


   private getAuthHeaders() {
    return { headers: new HttpHeaders({
        'Authorization': `Bearer ${this.token}`,
        'Content-Type': 'application/json'
      })
    };
```

```
  }

  addbook(entry : entry)
  {
    return
this.http.post<entry>(`${this.apiURL}/AddBooks`,entry,this.getAuthHeaders(
))
  }

getfavbook() {
  return this.http.get<Book[]>(`${this.apiURL}/GetallBooks`,
this.getAuthHeaders());
}
getAllBooks(): Observable<Book[]> {
    return this.http.get<Book[]>(`${this.apiURL}/GetallBooks`);
  }

deleteBook(bookId: number) {
  return this.http.delete(`${this.apiURL}/DeleteBook`, {
    params: { id: bookId.toString() },
    headers: new HttpHeaders({
      'Authorization': `Bearer ${this.token}`
    }),
     responseType: 'text'
  });
}

  updateBook(bookId: number, book: Book) {
    return this.http.put<Book>(`${this.apiURL}/UpdateBook/${bookId}`,
book, this.getAuthHeaders());
  }

getAllNewBooks() {
  return this.http.get<NewBook[]>(`${this.apiURL}/FetchAllNewBooks`,
this.getAuthHeaders());
}


  getfavplaces(): place[]
  {
```

```
    return [
      {Name : 'Valparai', altitude: 1800},
      {Name : 'Munnar', altitude: 2000},
      {Name : 'Dhimbam', altitude: 1500}
    ]
  }



}
```

```
export interface Book {
  bookId?: number;
  title: string;
  description: string;
  rating: number;
  price: number;
}
```